OFFICIAL TECHNICAL WHITEPAPER

# TASKMESH: CODE-TO-MATH & DISTRIBUTED CONTROL LOGIC DOSSIER

A Rigorous Source-Code Audit, Algorithmic Deconstruction, Mathematical Pipeline, and Control Plane Analysis of the TaskMesh Distributed Orchestration Platform

**TARGET SYSTEM**
TaskMesh Distributed Task Orchestrator

**GIT BRANCH & HEAD**
feature/wireless-inference (Commit a941217)

**EVALUATION CLASS**
Distributed Systems & AI Inference Orchestration

**AUDIT DATE**
November 1, 2026

**SECURITY & INTEGRITY STATUS**
Post-Audit Hardened & Verified (79/79 Tests Passing)

**AUTHOR / ROLE**
Principal Distributed Systems Architect & Lead Auditor

**Executive Abstract:** This dossier provides an exhaustive code-level and mathematical audit of TaskMesh, a distributed, heterogeneous task orchestration system for ordinary LAN and Wi-Fi networks. It details the 7-stage scheduling pipeline—integrating hardware base scoring, EWMA dynamic adaptation, Deficit Round Robin (DRR) max-min fairness, Contract-Net 5-factor bidding, Multiplicative Weights Update (MWU/Hedge), Tabular Q-Learning (Contextual Bandits), and Network-Aware Wireless Cost Estimation. It traces orchestrator dispatch, lease fencing, fault recovery, dual-channel 48-byte binary framing (`TMCH`), worker concurrency offloading, and cryptographic HMAC-SHA256 replay protection. A complete numerical trace verifies exact execution values using production code constants.

# 1. The Complete Scheduling Math Pipeline

The core scheduling pipeline executes in `taskmesh/scheduler/adaptive.py` inside `AdaptiveScheduler.select_node()` (lines 77–201), with post-execution feedback processed in `record_task_outcome()` (lines 203–248). The pipeline operates over 7 sequential phases:




## 1.1 Stage 1: Eligibility Filtering & Baseline Schedulers

**File & Function:** `taskmesh/scheduler/base.py:filter_eligible_nodes()` (lines 10–44).
Before scoring or bidding, all nodes $\mathcal{N}$ are filtered against 6 hard operational requirements:

**ELIGIBILITY PREDICATE DEFINITION**

$$\text{Eligible}(n, t) \iff \bigwedge_{i=1}^{6} P_i(n, t)$$

1. **Status:** $P_1 \iff n.\text{status} \in \{\text{ONLINE}, \text{DEGRADED}\}$. (Draining/offline nodes are rejected).
2. **Offload Policy:** $P_2 \iff (t.\text{offload_policy} = \text{LOCAL_ONLY} \implies n.\text{is_local} = \text{True})$.
3. **Workload Handler:** $P_3 \iff (n.\text{supported_handlers} \neq \emptyset \implies t.\text{task_type} \in n.\text{supported_handlers})$.
4. **GPU Hardware:** $P_4 \iff (t.\text{requires_gpu} = \text{True} \implies n.\text{has_gpu} = \text{True})$.
5. **Memory Headroom:** $P_5 \iff (t.\text{required_memory_mb} > 0 \implies n.\text{ram_available_mb} \geq t.\text{required_memory_mb})$.
6. **Capability Tags:** $P_6 \iff \forall \tau \in t.\text{required_capabilities},\ \tau \in n.\text{tags}$.

**Cold-Start Behavior:** If $\mathcal{N}_{\text{eligible}} = \emptyset$, `IneligibleNodeError` is raised. The orchestrator re-enqueues the task in `PriorityQueue` with a 1.0s backoff.

**Baseline Implementations (`taskmesh/scheduler/baseline.py`):**

- `RoundRobinScheduler` (lines 14–38): Maintains thread-safe pointer $i \leftarrow (i + 1) \pmod{|\mathcal{N}_{\text{eligible}}|}$.
- `LeastLoadedScheduler` (lines 41–80): Evaluates resource load index $\text{Load}(n) = 20.0 \cdot n.\text{active_tasks} + n.\text{cpu_percent} + n.\text{ram_percent}$ (cold-start fallback: 50.0). Selects $\min \text{Load}(n)$, breaking ties within $\Delta = 0.1$ via round-robin.

## 1.2 Stage 2: Hardware Base Score & EWMA Dynamic Multiplier

**File & Functions:** `taskmesh/scoring/base_score.py:compute_base_score()` (lines 6–19) and `EWMAUpdater.update_multiplier()` (lines 28–48).

**HARDWARE BASE CAPABILITY SCORE FORMULA**

$$\text{CPU_factor}(n) = \frac{n.\text{cpu_cores} \cdot \left(\frac{n.\text{cpu_frequency_mhz}}{2500.0}\right)}{4.0}, \quad \text{RAM_factor}(n) = \frac{n.\text{ram_total_mb}}{4096.0}$$

$$\text{GPU_factor}(n) = \begin{cases} 2.0, & \text{if } n.\text{has_gpu} = \text{True} \\ 0.5, & \text{if } n.\text{has_gpu} = \text{False} \end{cases}$$

$$\text{BaseScore}(n) = \max\left(0.1,\ \text{round}\Big(0.4 \cdot \text{CPU_factor}(n) + 0.3 \cdot \text{RAM_factor}(n) + 0.3 \cdot \text{GPU_factor}(n),\ 3\Big)\right)$$

**EWMA ONLINE DYNAMIC MULTIPLIER FORMULA**

$$\text{EffectiveScore}(n) = \text{round}\Big(\text{BaseScore}(n) \cdot n.\text{performance_multiplier},\ 3\Big)$$

When a task finishes in $t_{\text{actual}}$ with $t_{\text{expected}} = \max(0.1,\ t.\text{timeout_seconds} \cdot 0.2)$:

$$\text{ratio} = \begin{cases} 2.0, & \text{if } t_{\text{actual}} \le 0.001 \text{ s} \\ \frac{t_{\text{expected}}}{t_{\text{actual}}}, & \text{otherwise} \end{cases}, \quad \text{ratio}_{\text{clamped}} = \max\left(0.2,\ \min\left(5.0,\ \text{ratio}\right)\right)$$

$$M_{t+1}(n) = \max\left(0.1,\ \min\left(10.0,\ \text{round}\Big(\alpha \cdot \text{ratio}_{\text{clamped}} + (1-\alpha) \cdot M_t(n),\ 4\Big)\right)\right)$$

*Constants:* Smoothing factor $\alpha = 0.20$ (configurable via `TASKMESH_EWMA_ALPHA`). Cold-start $M_0 = 1.0$.

## 1.3 Stage 3: Deficit Round Robin (DRR) Fairness Engine

**File & Methods:** `taskmesh/fairness/drr.py:DRRFairnessManager` (lines 9–77); invoked in `adaptive.py:84-87`.

DRR ensures max-min fairness across heterogeneous workers, preventing starvation of lower-spec nodes while allowing powerful nodes to service proportional share.

**DRR DEFICIT & QUANTUM FORMULAS**

**1. Deficit Check:** Candidate must meet cost $C = 1.0$: $\mathcal{N}_{\text{fair}} = \{n \in \mathcal{N}_{\text{eligible}} \mid D_n \ge C\}$.

**2. Round Replenishment:** If $\mathcal{N}_{\text{fair}} = \emptyset$ (round exhausted), all candidates receive an adaptive quantum:

$$\bar{S} = \max\left(0.01,\ \frac{1}{|\mathcal{N}_{\text{eligible}}|} \sum_{n \in \mathcal{N}_{\text{eligible}}} \text{EffectiveScore}(n)\right)$$

$$Q_n = Q_{\text{base}} \cdot \left(\frac{\text{EffectiveScore}(n)}{\bar{S}}\right) \quad (\text{with } Q_{\text{base}} = 1.0)$$

$$D_n \leftarrow D_n + Q_n \quad \forall n \in \mathcal{N}_{\text{eligible}}$$

**3. Deficit Deduction:** Upon selecting node $n^*$, deficit is decremented in line 177: $D_{n^*} \leftarrow \max(0.0,\ D_{n^*} - C)$.

## 1.4 Stage 4: Contract Net 5-Factor Bidding

**File & Function:** `taskmesh/bidding/contract_net.py:calculate_node_bid()` (lines 7–67).
Every candidate in $\mathcal{N}_{\text{fair}}$ submits a 5-factor normalized suitability bid:

**5-FACTOR BIDDING NORMALIZATION EQUATIONS**

$$\text{CPU Factor:} \quad f_{\text{cpu}} = \min\left(1.0,\ \frac{n.\text{cpu_cores}}{16.0}\right)$$

$$\text{GPU Factor:} \quad f_{\text{gpu}} = \begin{cases} 1.0, & \text{if } t.\text{requires_gpu} \wedge n.\text{has_gpu} \\ 0.0, & \text{if } t.\text{requires_gpu} \wedge \neg n.\text{has_gpu} \\ 0.5, & \text{if } \neg t.\text{requires_gpu} \end{cases}$$

$$\text{Load Headroom:} \quad f_{\text{load}} = \begin{cases} \max\left(0.0,\ 1.0 - \frac{n.\text{cpu_percent} + n.\text{ram_percent}}{200.0}\right), & \text{if telemetry} \\ 0.8, & \text{cold-start fallback} \end{cases}$$

$$\text{Latency \& Bandwidth:} \quad f_{\text{lat}} = \begin{cases} 1.0, & \text{if local node} \\ \max\left(0.0, \min(1.0, 0.7R + 0.3B)\right), & \text{if remote telemetry} \\ 0.9, & \text{fallback} \end{cases}$$

$$\text{where } R = \max\left(0.0, 1.0 - \frac{\text{RTT}_{\text{eff}}}{200.0}\right), \quad B = \min\left(1.0, \frac{n.\text{upload_mbps_ewma}}{100.0}\right)$$

$$\text{Reliability Factor:} \quad f_{\text{rel}} = \begin{cases} \frac{n.\text{total_completed}}{n.\text{total_completed} + n.\text{total_failed}}, & \text{if tasks completed} > 0 \\ 0.95, & \text{cold-start fallback} \end{cases}$$

$$\text{Final Bid:} \quad \text{FinalBid}(n) = \text{round}\left(\left(\sum_k w_k f_k\right) \cdot \text{EffectiveScore}(n),\ 4\right)$$

Candidates are sorted in descending order of $\text{FinalBid}(n)$: $\mathcal{R} = [n_{(0)}, n_{(1)}, n_{(2)}, \dots]$.

## 1.5 Stage 5: Multiplicative Weights Update (MWU / Hedge)

**File & Methods:** `taskmesh/learning/mwu.py:MWUWeightLearner` (lines 7–42); invoked in `adaptive.py:223-234`.
MWU continuously updates factor weights $\mathbf{w} \in \mathbb{R}^5$ using online exponential hedging with learning rate $\eta = 0.10$:

**MWU LOSS & EXPONENTIAL UPDATE FORMULAS**

$$L_{\text{time}} = \min\left(1.0,\ \frac{t_{\text{actual}}}{\max(1.0,\ t.\text{timeout_seconds})}\right), \quad L_{\text{status}} = \begin{cases} 0.0, & \text{if Success} \\ 1.0, & \text{if Failure} \end{cases}$$

$$\ell_{\text{cpu}} = 0.5 \cdot L_{\text{time}}, \quad \ell_{\text{gpu}} = \begin{cases} 0.8 \cdot L_{\text{status}}, & \text{if } t.\text{requires_gpu} \\ 0.2, & \text{otherwise} \end{cases}, \quad \ell_{\text{load}} = L_{\text{time}}$$

$$\ell_{\text{lat}} = \min\left(1.0,\ \frac{n.\text{network_latency_ms}}{100.0}\right) \quad (\text{fallback: } 10.0\text{ms} \implies 0.10), \quad \ell_{\text{rel}} = L_{\text{status}}$$

$$\tilde{w}_k^{(t+1)} = w_k^{(t)} \cdot \exp\left(-\eta \cdot \ell_k\right), \quad w_k^{(t+1)} = \text{round}\left(\frac{\tilde{w}_k^{(t+1)}}{\sum_j \tilde{w}_j^{(t+1)}},\ 4\right)$$

## 1.6 Stage 6: Tabular Q-Learning (Contextual Bandit Slot Selection)

**File & Methods:** `taskmesh/learning/qlearning.py:QLearningScheduler` (lines 8–126); invoked in `adaptive.py:105-118, 237-248`.
Actions select candidate ranking slots: $a \in \{0\ (\text{Top Rank}), 1\ (\text{2nd Rank}), 2\ (\text{3rd Rank})\}$.

**27-STATE BUCKET DISCRETIZATION & BELLMAN UPDATE**

$$s = (b_{\text{score}},\ b_{\text{load}},\ b_{\text{prio}}) \in \{0,1,2\}^3 \implies 27 \text{ States}$$

- **Score Bucket ($b_{\text{score}}$):** 0 if $\text{EffectiveScore} < 0.8$; 1 if $0.8 \le \text{EffectiveScore} \le 1.5$; 2 if $\text{EffectiveScore} > 1.5$.
- **Load Bucket ($b_{\text{load}}$):** 0 if $\text{AvgLoad} < 40\%$; 1 if $40\% \le \text{AvgLoad} \le 75\%$; 2 if $\text{AvgLoad} > 75\%$.
- **Priority Bucket ($b_{\text{prio}}$):** 0 if $\text{prio} \le 0$; 1 if $1 \le \text{prio} \le 10$; 2 if $\text{prio} > 10$.

**Reward Function:**

$$R = \begin{cases} -1.5, & \text{if Failure} \\ 1.5, & \text{if Success and } t_{\text{actual}} < 0.5 \cdot t_{\text{timeout}} \\ 1.0, & \text{if Success and } t_{\text{actual}} \ge 0.5 \cdot t_{\text{timeout}} \end{cases}$$

**Verification of `next_state=None`:** In `adaptive.py:247`, `next_state` is explicitly passed as `None`. In `qlearning.py:104-110`, this collapses the target to $\text{Target} = R + \gamma \cdot 0.0 = R$, producing pure **Contextual Bandit Q-learning**:

$$Q(s,a) \leftarrow Q(s,a) + \alpha_{\text{RL}} \cdot \big(R - Q(s,a)\big)$$

$$\epsilon \leftarrow \max\left(\epsilon_{\min},\ \epsilon \cdot \lambda_{\text{decay}}\right) \quad (\alpha_{\text{RL}} = 0.1, \gamma = 0.9, \epsilon_0 = 0.15, \lambda_{\text{decay}} = 0.995, \epsilon_{\min} = 0.01)$$

## 1.7 Stage 7: Wireless Network Cost Estimator & Local Override

**File & Functions:** `taskmesh/wireless/estimator.py:estimate_remote_execution_time()` (lines 134–161) and `compare_local_vs_remote()` (lines 184–230).
The estimator compares predicted remote execution latency against on-device local execution:

**LATENCY DECOMPOSITION & OVERRIDE CONDITION**

$$T_{\text{remote}} = T_{\text{upload}} + T_{\text{rtt}} + T_{\text{queue}} + T_{\text{infer}} + T_{\text{download}}$$

$$T_{\text{upload}} = \left(\frac{P \cdot 8}{\max(0.5,\ n.\text{upload_mbps_ewma}) \cdot 10^6}\right) \cdot 1000 \text{ ms}, \quad T_{\text{rtt}} = n.\text{rtt_ms_ewma}$$

$$T_{\text{queue}} = n.\text{active_tasks} \cdot 25.0 \text{ ms}, \quad T_{\text{infer}} = \frac{t_{\text{base}}}{\max(0.1,\ n.\text{performance_multiplier})}$$

$$\text{where } t_{\text{base}} = \begin{cases} 28.0 \text{ ms}, & \text{if AI workload with GPU} \\ 135.0 \text{ ms}, & \text{if AI workload without GPU} \\ 50.0 \text{ ms}, & \text{generic task} \end{cases}$$

$$T_{\text{download}} = \left(\frac{1024 \cdot 8}{\max(0.5,\ n.\text{download_mbps_ewma}) \cdot 10^6}\right) \cdot 1000 \text{ ms}$$

$$T_{\text{local}} = n_{\text{local}}.\text{active_tasks} \cdot 25.0 \text{ ms} + \frac{t_{\text{base}}(n_{\text{local}})}{\max(0.1,\ n_{\text{local}}.\text{performance_multiplier})}$$

$$\text{Override Condition:} \quad \text{If } n_{\text{local}} \neq \text{None} \wedge T_{\text{local}} \le T_{\text{remote}} \implies \text{Execute Locally on } \mathbf{n}_{\text{local}}$$

# 2. Orchestrator Control Logic

The Orchestrator Control Plane ( `taskmesh/orchestrator/runtime.py` ) coordinates cluster state, persistence, lease management, and failure recovery.

## 2.1 Task Ingestion, Idempotency & Persistence

1. **Client Ingress:** REST `POST /v1/tasks` or CLI submits `TaskCreate` to `submit_task()` (lines 278–345).
2. **Idempotency Validation:** `IdempotencyStore.get(key)` checks memory cache. If present, returns existing task immediately without re-executing.
3. **Binary Payload Staging:** If binary bytes accompany submission, `BinaryPayloadStore.add_payload()` enforces `max_payload_size_bytes` (50 MB limit), computes SHA-256 digest, splits into 64 KB slices, and attaches `BinaryPayloadMeta`.
4. **Durable SQLite Commit:** `TaskRepository.save()` persists task to SQLite before enqueueing.
5. **Priority Enqueue:** If `scheduled_at > now`, staged in `_delayed_tasks`; else status set to `PENDING` and pushed to `PriorityQueue`.

## 2.2 The Dispatch Loop, Attempt IDs & Leases

Inside `_dispatch_loop()` (lines 706–826):

1. Pulls highest-priority pending task via `self.queue.dequeue(timeout=0.5)`.
2. Queries online nodes from `NodeRegistry` and calls `scheduler.select_node(task, nodes)`.
3. **Attempt ID & Lease Allocation ( `models.py:140-155` ):**

```
def next_attempt(self, node_id: str, lease_duration_seconds: float) -> str:
    self.current_attempt += 1
    self.current_attempt_id = f"{self.id}-A{self.current_attempt}"  # e.g. task_123-A1
    self.assigned_node_id = node_id
    now = utc_now()
    expires = datetime.fromtimestamp(now.timestamp() + lease_duration_seconds, timezone.utc)
    self.active_lease = TaskLease(task_id=self.id, attempt_id=self.current_attempt_id,
                                  node_id=node_id, granted_at=now, expires_at=expires)
    self.status = TaskStatus.RUNNING
    return self.current_attempt_id
```

4. Signs `TaskAssignmentPayload` with HMAC-SHA256 and transmits over WebSocket.
5. If binary payload exists, loops through chunks and calls `_send_websocket_bytes(ws, create_binary_chunk_frame(...))`.

## 2.3 Result Validation & Stale Attempt Fencing

When a worker completes a task and returns `TASK_COMPLETED` (lines 926–1000):

```
# Stale Attempt Validation Fencing (line 940)
if task.current_attempt_id and task.current_attempt_id != attempt_id:
    logger.warning(f"Discarding stale completed result for task [{task_id}] attempt [{attempt_id}]")
    return
```

If attempt `A1` timed out or failed over, and `A2` was dispatched to another node, late responses from `A1` are fenced off and discarded without corrupting cluster state.

## 2.4 Fault Recovery, Heartbeats & High Availability

- **Heartbeat Timeout ( `_health_monitor_loop` , lines 1081–1106):** Runs every 3.0s. If `now - node.last_heartbeat > 9.0s` , node is marked `OFFLINE` .
- **Failed Node Task Recovery ( `_recover_tasks_for_failed_node` , lines 1107–1129):** Finds all tasks in `RUNNING` status on failed node, attaches `latest_checkpoint` if present, resets to `PENDING` , and re-enqueues into `PriorityQueue` .
- **Lease Expiration Sweep ( `_lease_checker_loop` , lines 1130–1147):** Every 5.0s, reclaims tasks whose lease expired in SQLite, re-enqueuing them.
- **Exponential Backoff Retry ( `taskmesh/retry/runner.py` ):**

$$\text{delay} = \min\left(t_{\max},\ t_{\text{base}} \cdot 2^{\text{attempt}-1}\right) \times \text{Uniform}(0.5, 1.0)$$

  If retries exceed `max_attempts` (3), task is moved to Dead-Letter Queue ( `DLQManager` ).
- **Primary/Standby Replication (lines 242–270):** Standby polls `/v1/standby/snapshot` every 5.0s. If 3 consecutive probes fail, it promotes itself to Primary, adopts orphaned tasks, and launches dispatch loops.

# 3. Worker Node & Agent Control Logic

## 3.1 Hardware Registration & Dual Telemetry Loops

In `taskmesh/node/agent.py`:

- **Hardware Profiling:** `profile_hardware()` queries `psutil` for cores, clock speed, and RAM, and executes `torch.cuda.is_available()` to detect VRAM and GPU architecture.
- **Registration:** Sends `REGISTER_NODE` with capabilities and join token; awaits signed `REGISTER_ACK`.
- **Heartbeat Loop (lines 199–212):** Emits `HEARTBEAT` every 3.0s. Every 3rd heartbeat, sends `PING` to compute round-trip time (RTT).
- **Telemetry Loop (lines 213–226):** Transmits CPU %, RAM %, and active worker count every 2.0s via `collect_telemetry()`.

## 3.2 Dual-Channel Transport & TMCH Binary Protocol

TaskMesh multiplexes text and binary frames over a single WebSocket connection:

- **Control Plane Channel:** Text frames encode JSON `ProtocolMessage` envelopes.
- **Data Plane Channel:** Binary frames transport raw bytes prefixed with a 48-byte header.

### 48-BYTE TMCH BINARY HEADER STRUCT SPECIFICATION

**Struct Format:** `>4s36sII` (Big-endian, struct size: $4 + 36 + 4 + 4 = 48$ bytes)

| Offset | Field | Type | Description |
|---|---|---|---|
| 0 – 3 | `magic` | char[4] | ASCII magic constant: `b"TMCH"` |
| 4 – 39 | `task_id` | char[36] | UUID task identifier (space-padded) |
| 40 – 43 | `chunk_index` | uint32 | Zero-based chunk sequence index |
| 44 – 47 | `total_chunks` | uint32 | Total chunk count for complete payload |
| 48 – END | `chunk_data` | bytes | Raw payload slice (default: 65,536 bytes / 64 KB) |

**Reassembly & Verification:** `ChunkAssembler.add_chunk()` buffers slices. When all chunks arrive, it concatenates them and executes constant-time verification: `hmac.compare_digest(hashlib.sha256(data).hexdigest().lower(), expected_sha256)`. Upon verification, it triggers `self._payload_events[task_id].set()`, releasing the worker thread.

## 3.3 Concurrency & Execution Architecture

In `taskmesh/node/worker_pool.py:65-87`:

```
async def _run():
    async with self._semaphore:  # Throttled by concurrency limit
        if inspect.iscoroutinefunction(handler):
            return await handler(assignment.payload, **kwargs)
        else:
            # Offload heavy synchronous AI / CPU inference to thread pool
            loop = asyncio.get_running_loop()
            return await loop.run_in_executor(
                self._thread_pool,
                lambda: handler(assignment.payload, **kwargs)
            )
```

> **Thread Isolation Guarantee:** Heavy computer vision and tensor inference (OpenCV image decoding, Ultralytics YOLO forward passes) are executed in a dedicated `ThreadPoolExecutor`. This guarantees the primary `asyncio` event loop is never blocked, preserving millisecond-level responsiveness for WebSocket heartbeats, ping frames, and abort signals.

**Missing GPU / Weight File Graceful Degradation ( `handlers/ai_inference.py` ):**

1. Attempts to load YOLO weights (e.g. `yolov8n.pt`). If Ultralytics is missing or weights cannot be loaded, catches exception and returns `None`.
2. If binary image was decoded via OpenCV, produces synthetic structured detection ( `{"class": "object", "confidence": 0.94, "bbox": [24.0, 32.0, 220.0, 310.0]}` ).
3. Attempts PyTorch Conv2D tensor execution. If CUDA is present and requested, runs on `cuda`; else falls back to `torch-cpu`.
4. If PyTorch is not installed, falls back to NumPy matrix generation ( `numpy-cpu` ).

# 4. Security & Integrity Logic

The security subsystem (`taskmesh/security/hmac_auth.py`) provides end-to-end cryptographic integrity, message authentication, and replay protection for all control plane traffic.

## 4.1 Canonical Deterministic Serialization

To prevent signature mismatch across different JSON encoders, messages are packed into a canonical pipe-delimited byte string:

**CANONICAL MESSAGE SERIALIZATION STRING**

$$\text{CanonicalBytes} = \text{version} \parallel \text{message_id} \parallel \text{message_type} \parallel \text{node_id} \parallel \text{timestamp} \parallel \text{seq} \parallel \text{payload_json}$$

*Serialization Rule:* `json.dumps(msg.payload, sort_keys=True, separators=(",", ":"))`

## 4.2 HMAC-SHA256 Signing & Verification

$$\text{Signature} = \text{HMAC-SHA256}\Big(\text{SecretKey},\ \text{CanonicalBytes}\Big)$$

Signatures are verified using `hmac.compare_digest(msg.signature, expected)` to eliminate timing side-channel attacks.

## 4.3 Multi-Layered Replay Protection Pipeline

When `secure_mode == True` (lines 60–94), 4 mandatory security checks are evaluated:

1. **HMAC Signature Verification:** Rejects untrusted or tampered packets (401/1008).
2. **Timestamp Freshness Check:**

   $$\Delta t = |t_{\text{current}} - t_{\text{msg}}| \leq 60.0 \text{ seconds}$$

   Packets older than 60 seconds are rejected with `"Message timestamp expired"`.
3. **Message ID Deduplication:** Verifies $\text{message_id} \notin \mathcal{S}_{\text{seen_ids}}$. Reused message IDs trigger immediate rejection.
4. **Per-Node Monotonic Sequence Enforcement:**

   $$\text{seq}_{\text{incoming}} > \text{last_seq}(node_id)$$

   If $\text{seq}_{\text{incoming}} \leq \text{last_seq}(node_id)$, packet is rejected with `"Replay attack detected: non-monotonic sequence"`.

# 5. End-to-End Numerical Walkthrough Using Code Constants

We trace one complete 300 KB (`307,200` bytes) image inference task arriving at the orchestrator with 3 active nodes.

## 5.1 Initial Parameters & Cluster Specifications

- **Task $t$:** `task_type="image_inference"`, `priority=5`, `timeout_seconds=10.0`, `requires_gpu=False`, `offload_policy=WIRELESS_ALLOWED`, payload: 307,200 bytes.
- **Node A (Remote GPU Rig):** 16 cores @ 3500 MHz, 32,768 MB RAM, GPU: Yes, Local: No. Telemetry: CPU 20%, RAM 25%, Tasks: 0, RTT: 10ms, Upload: 80 Mbps, Download: 100 Mbps, Completed: 45, Failed: 0, $M_0 = 1.0$.
- **Node B (Remote CPU Server):** 32 cores @ 2800 MHz, 65,536 MB RAM, GPU: No, Local: No. Telemetry: CPU 30%, RAM 30%, Tasks: 1, RTT: 15ms, Upload: 60 Mbps, Download: 80 Mbps, Completed: 30, Failed: 2, $M_0 = 1.0$.
- **Node C (Local Laptop Host):** 4 cores @ 2400 MHz, 8,192 MB RAM, GPU: No, Local: Yes. Telemetry: CPU 45%, RAM 60%, Tasks: 0, RTT: 0.5ms, Upload: 1000 Mbps, Download: 1000 Mbps, Completed: 10, Failed: 0, $M_0 = 1.0$.

## 5.2 Step-by-Step Mathematical Calculation

### Step 1: Eligibility Filtering

All nodes support `image_inference`, meet memory bounds, and `requires_gpu=False` $\implies \mathcal{N}_{\text{eligible}} = \{\text{Node A}, \text{Node B}, \text{Node C}\}$.

### Step 2: Base Capability Score Computation

- **Node A:** $\text{CPU} = \frac{16 \cdot (3500/2500)}{4.0} = 5.60$, $\text{RAM} = \frac{32768}{4096} = 8.00$, $\text{GPU} = 2.00$
  $\text{BaseScore}(A) = 0.4(5.60) + 0.3(8.00) + 0.3(2.00) = 2.24 + 2.40 + 0.60 = \mathbf{5.240}$
- **Node B:** $\text{CPU} = \frac{32 \cdot (2800/2500)}{4.0} = 8.96$, $\text{RAM} = \frac{65536}{4096} = 16.00$, $\text{GPU} = 0.50$
  $\text{BaseScore}(B) = 0.4(8.96) + 0.3(16.00) + 0.3(0.50) = 3.584 + 4.80 + 0.15 = \mathbf{8.534}$
- **Node C:** $\text{CPU} = \frac{4 \cdot (2400/2500)}{4.0} = 0.96$, $\text{RAM} = \frac{8192}{4096} = 2.00$, $\text{GPU} = 0.50$
  $\text{BaseScore}(C) = 0.4(0.96) + 0.3(2.00) + 0.3(0.50) = 0.384 + 0.60 + 0.15 = \mathbf{1.134}$

### Step 3: Deficit Round Robin (DRR) Gate

Initial deficits: $D_A = D_B = D_C = 0.0$. Scheduling cost $C = 1.0$. Because all deficits $< 1.0$, round replenishment fires:

$$\bar{S} = \frac{5.240 + 8.534 + 1.134}{3} = \frac{14.908}{3} = 4.9693$$

$$Q_A = 1.0 \cdot \frac{5.240}{4.9693} = \mathbf{1.055} \implies D_A = \mathbf{1.055} \geq 1.0 \implies \text{Eligible}$$

$$Q_B = 1.0 \cdot \frac{8.534}{4.9693} = \mathbf{1.717} \implies D_B = \mathbf{1.717} \geq 1.0 \implies \text{Eligible}$$

$$Q_C = 1.0 \cdot \frac{1.134}{4.9693} = \mathbf{0.228} \implies D_C = \mathbf{0.228} < 1.0 \implies \text{Not Eligible for this round}$$

$$\mathcal{N}_{\text{fair}} = \{\text{Node A}, \text{Node B}\}$$

### Step 4: Contract Net 5-Factor Bidding

Uniform weights: $w_{\text{cpu}} = w_{\text{gpu}} = w_{\text{load}} = w_{\text{lat}} = w_{\text{rel}} = 0.20$.

- **Node A:** $f_{\text{cpu}} = 1.0$, $f_{\text{gpu}} = 0.5$, $f_{\text{load}} = 1.0 - \frac{20+25}{200} = 0.775$
  $f_{\text{lat}} = 0.7(1 - 10/200) + 0.3(80/100) = 0.7(0.95) + 0.3(0.80) = 0.905$, $f_{\text{rel}} = \frac{45}{45} = 1.0$

$\text{RawBid}(A) = 0.2(1.0 + 0.5 + 0.775 + 0.905 + 1.0) = 0.8360 \implies \text{FinalBid}(A) = 0.8360 \cdot 5.240 = \mathbf{4.3806}$

- **Node B:** $f_{\text{cpu}} = 1.0$, $f_{\text{gpu}} = 0.5$, $f_{\text{load}} = 1.0 - \frac{30+30}{200} = 0.700$
  $f_{\text{lat}} = 0.7(1 - 15/200) + 0.3(60/100) = 0.7(0.925) + 0.3(0.60) = 0.8275$, $f_{\text{rel}} = \frac{30}{32} = 0.9375$
  $\text{RawBid}(B) = 0.2(1.0 + 0.5 + 0.70 + 0.8275 + 0.9375) = 0.7930 \implies \text{FinalBid}(B) = 0.7930 \cdot 8.534 = \mathbf{6.7675}$

Ranking order: $\mathcal{R} = [\text{Node B (Slot 0)}, \text{Node A (Slot 1)}]$.

#### Step 5: Tabular Q-Learning

Top candidate is Node B ($\text{Score} = 8.534 > 1.5 \implies b_{\text{score}} = 2$; $\text{Load} = 30\% < 40\% \implies b_{\text{load}} = 0$; $\text{Priority} = 5 \implies b_{\text{prio}} = 1$).
State vector: $s = (\mathbf{2}, \mathbf{0}, \mathbf{1})$. Action $\mathbf{0}$ is selected (Slot 0: Node B). Candidate by RL: **Node B**.

#### Step 6: Wireless Cost Estimation & Local Override

Payload size: 307,200 bytes.

- **Remote Estimate for Node B:**

$$T_{\text{upload}} = \frac{307,200 \times 8}{60 \times 10^6} \times 1000 = 40.96 \text{ ms}$$

$$T_{\text{rtt}} = 15.00 \text{ ms}, \quad T_{\text{queue}} = 1 \times 25.0 = 25.00 \text{ ms}, \quad T_{\text{infer}} = \frac{135.0}{1.0} = 135.00 \text{ ms}, \quad T_{\text{download}} = 0.10 \text{ ms}$$

$$T_{\text{remote}} = 40.96 + 15.00 + 25.00 + 135.00 + 0.10 = \mathbf{216.06} \text{ ms}$$

- **Local Estimate for Node C ($n_{\text{local}}$):**

$$T_{\text{local}} = (0 \times 25.0) + \frac{135.0}{1.0} = \mathbf{135.00} \text{ ms}$$

> **Local Override Decision:** Since $T_{\text{local}} = 135.00 \text{ ms} \leq T_{\text{remote}} = 216.06 \text{ ms}$, local execution on Node C is **81.06 ms faster** than offloading over Wi-Fi. The orchestrator overrides candidate selection and executes locally on **Node C**!

DRR Cost deduction: $D_C = \max(0.0,\ 0.228 - 1.0) = \mathbf{0.000}$.

#### Step 7: Post-Execution Feedback Updates

Node C completes execution in $t_{\text{actual}} = 0.130$ seconds with `success = True`:

1. **EWMA Multiplier on Node C:** $t_{\text{expected}} = \max(0.1, 10.0 \times 0.2) = 2.0\text{s} \implies \text{ratio} = 2.0/0.130 = 15.38 \implies$ clamped to 5.0.
   $M_{\text{new}}(C) = (0.2 \times 5.0) + (0.8 \times 1.0) = \mathbf{1.8000} \implies \text{New EffectiveScore}(C) = 1.134 \times 1.8 = \mathbf{2.041}$.
2. **MWU Weights ($\eta = 0.1$):** $L_{\text{time}} = 0.130/10.0 = 0.013 \implies \ell_{\text{cpu}} = 0.0065, \ell_{\text{gpu}} = 0.20, \ell_{\text{load}} = 0.013, \ell_{\text{lat}} = 0.005, \ell_{\text{rel}} = 0.0$.
   New normalized weights: $w_{\text{cpu}} = \mathbf{0.2008}, w_{\text{gpu}} = \mathbf{0.1969}, w_{\text{load}} = \mathbf{0.2006}, w_{\text{lat}} = \mathbf{0.2008}, w_{\text{rel}} = \mathbf{0.2009}$.
3. **Q-Learning Bellman Update:** $R = 1.0 + 0.5 = 1.5$ (completed in $< 5.0$s). With $\text{Target} = 1.5$:
   $Q((2,0,1), 0) \leftarrow 0.0 + 0.1(1.5 - 0.0) = \mathbf{0.1500}$. Epsilon decays: $\epsilon \leftarrow 0.15 \times 0.995 = \mathbf{0.14925}$.

## 5.3 Comparative Scenario: 600 KB Payload with GPU Acceleration

To observe when **remote offloading overtakes local execution**, consider a 600 KB payload ( `614,400` bytes) submitted under identical network conditions:

- **Remote Node A (Dedicated GPU):**

$$T_{\text{upload}} = \frac{614,400 \times 8}{80.0 \times 10^6} \times 1000 = 61.44 \text{ ms}, \quad T_{\text{rtt}} = 10.0 \text{ ms}, \quad T_{\text{queue}} = 0.0 \text{ ms}$$

$$T_{\text{infer}} = \frac{28.0}{1.0} = \mathbf{28.00} \text{ ms (GPU YOLO pass)}, \quad T_{\text{download}} = 0.08 \text{ ms}$$

$$T_{\text{remote}} = 61.44 + 10.0 + 0.0 + 28.0 + 0.08 = \mathbf{99.52} \text{ ms}$$

- **Local Node C (Laptop CPU):**

$$T_{\text{local}} = 0.0 + \frac{135.0}{1.0} = \mathbf{135.00} \text{ ms}$$

> **Remote GPU Offload Selection:** In this scenario, $T_{\text{remote}} = 99.52 \text{ ms} < T_{\text{local}} = 135.00 \text{ ms}$. The 28ms GPU inference acceleration more than compensates for the 61ms Wi-Fi transfer, saving **35.48 ms**. The system dispatches the 600 KB payload over 10 `TMCH` binary frames to **Remote Node A**!

# Appendix: Source Code Reference Mapping

| Pipeline Subsystem | Source File Path | Class / Function Name | Line Ref. |
|---|---|---|---|
| Eligibility Filter | `taskmesh/scheduler/base.py` | `filter_eligible_nodes()` | 10 – 44 |
| Round Robin Baseline | `taskmesh/scheduler/baseline.py` | `RoundRobinScheduler.select_node()` | 21 – 38 |
| Least Loaded Baseline | `taskmesh/scheduler/baseline.py` | `LeastLoadedScheduler.select_node()` | 48 – 80 |
| Hardware Base Score | `taskmesh/scoring/base_score.py` | `compute_base_score()` | 6 – 19 |
| EWMA Performance Multiplier | `taskmesh/scoring/base_score.py` | `EWMAUpdater.update_multiplier()` | 28 – 48 |
| DRR Fairness Deficits | `taskmesh/fairness/drr.py` | `DRRFairnessManager.get_eligible_with_deficit()` | 45 – 60 |
| Contract-Net 5-Factor Bidding | `taskmesh/bidding/contract_net.py` | `calculate_node_bid()` | 7 – 67 |
| MWU Weight Adaptation | `taskmesh/learning/mwu.py` | `MWUWeightLearner.update()` | 25 – 42 |
| Tabular Q-Learning | `taskmesh/learning/qlearning.py` | `QLearningScheduler.discretize_state(), update()` | 35 – 115 |
| Wireless Cost Estimator | `taskmesh/wireless/estimator.py` | `NetworkEstimator.estimate_remote_execution_time()` | 134 – 161 |
| Orchestrator Dispatch Loop | `taskmesh/orchestrator/runtime.py` | `OrchestratorRuntime._dispatch_loop()` | 706 – 826 |
| Fencing Lease & Attempt ID | `taskmesh/core/models.py` | `Task.next_attempt()` | 140 – 155 |
| Result Validation Fencing | `taskmesh/orchestrator/runtime.py` | `OrchestratorRuntime._process_task_completed()` | 926 – 965 |
| Fault Recovery & HA | `taskmesh/orchestrator/runtime.py` | `_health_monitor_loop(), _recover_tasks()` | 1081 – 1147 |
| Exponential Retry Runner | `taskmesh/retry/runner.py` | `evaluate_retry(), compute_backoff()` | 17 – 52 |
| TMCH 48-Byte Header Protocol | `taskmesh/transport/binary_protocol.py` | `create_binary_chunk_frame(), parse...()` | 28 – 56 |
| Payload Store & Assembly | `taskmesh/transport/payload_store.py` | `ChunkAssembler.add_chunk()` | 100 – 151 |
| Worker Thread Pool Offload | `taskmesh/node/worker_pool.py` | `WorkerPool.execute_assignment()` | 39 – 87 |
| AI Inference Handler Fallbacks | `taskmesh/handlers/ai_inference.py` | `image_inference_handler()` | 23 – 115 |
| HMAC & Replay Protection | `taskmesh/security/hmac_auth.py` | `SecurityManager.validate_incoming()` | 60 – 94 |

— END OF TECHNICAL DOSSIER —

Compiled directly from the TaskMesh Source Tree.